# 任务 1：数据库与查询条件基础说明

1.页面中的输入为什么会与数据库查询有关
电商平台的所有商品信息（名称、价格、库存等）都存储在后台数据库里。用户在搜索框输入 “无线耳机” 时，页面无法直接获取这些数据，必须把输入内容传给后台，后台再向数据库发起查询，才能找到匹配的商品信息并返回给页面展示。所以页面输入本质上是用户向数据库发起查询的 “触发条件”。

1. 一个最小业务链路是什么

用户输入搜索关键词 → 后台接收输入并构造 SQL 查询 → 数据库执行查询并

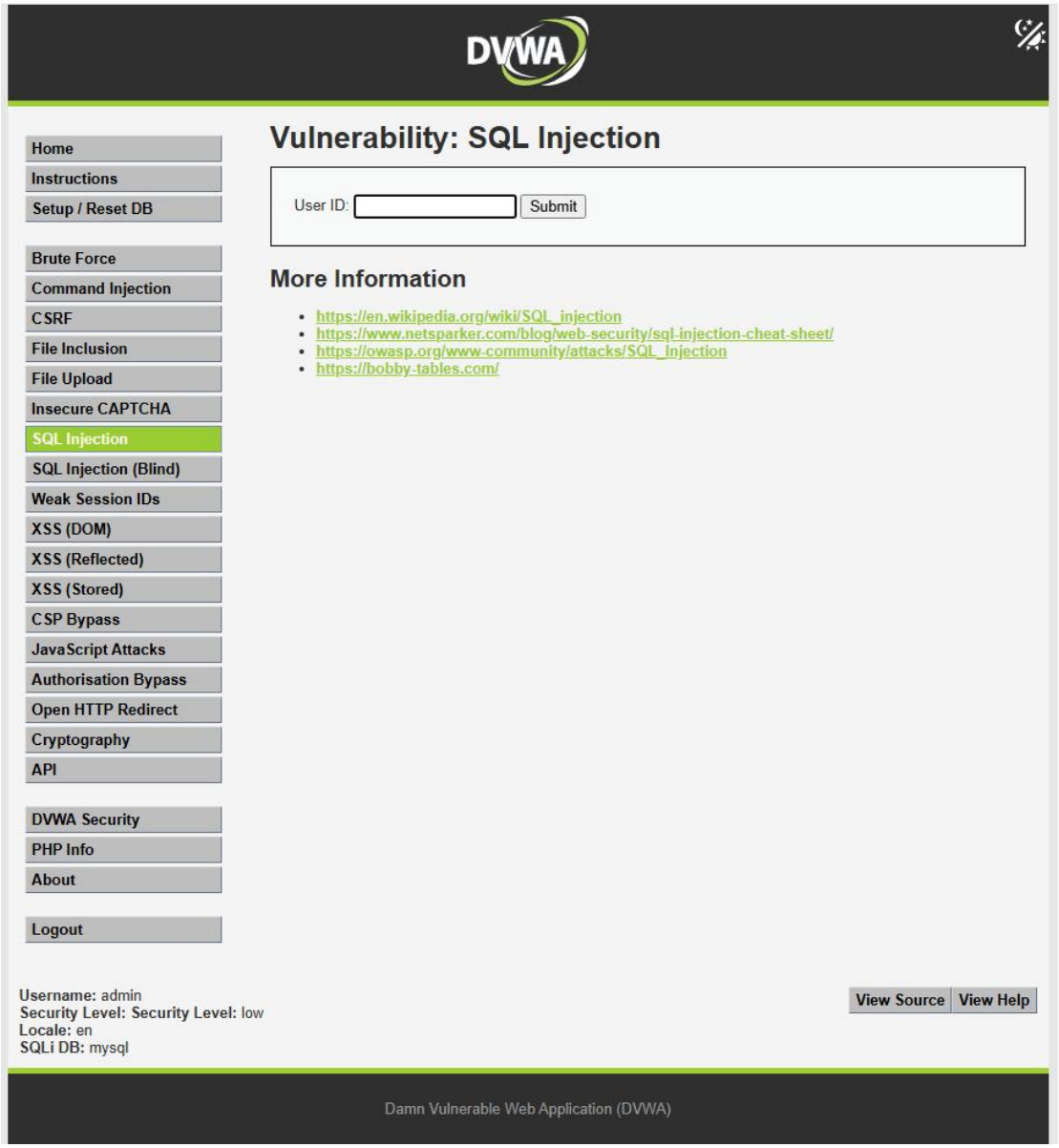
返回结果 → 后台处理结果并渲染到页面 → 用户看到商品列表

2. 用户输入是如何进入查询条件的
后台程序会直接把用户输入的内容，拼接到 SQL 查询语句中。比如用户输入的关键词 “无线耳机”，会被拼接到 SQL 语句的条件部分：SELECT＊FROM goods WHERE name LIKE ’%用户输入的关键词%’;这里用户输入的 “无线耳机” 就直接成为了 SQL 查询的条件参数，决定了数据库要返回哪些数据。

3. 为什么这为 SQL 注入创造了可能性

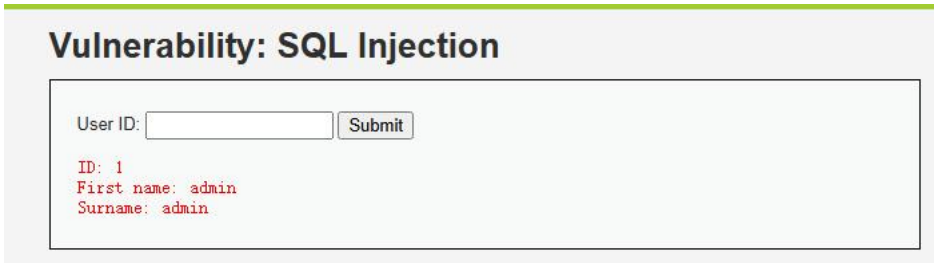
如果后台程序没有对用户输入进行过滤和验证，攻击者就可以在输入框中构造恶意 SQL 语句，改变原查询的逻辑。
比如用户输入 ’ OR 1=1 --，拼接到 SQL 后会变成：
SELECT * FROM goods WHERE name LIKE ’%’ OR 1=1 --%’;
其中 OR 1=1 会让查询条件永远为真，--会注释掉后面的语句，导致数据库返回所有商品数据，甚至被进一步利用来窃取、篡改数据。这种直接拼接用户输入到 SQL 语句的方式，就是 SQL 注入产生的根本原因。

# 任务 2：DVWA SQL Injection Low 现象记录



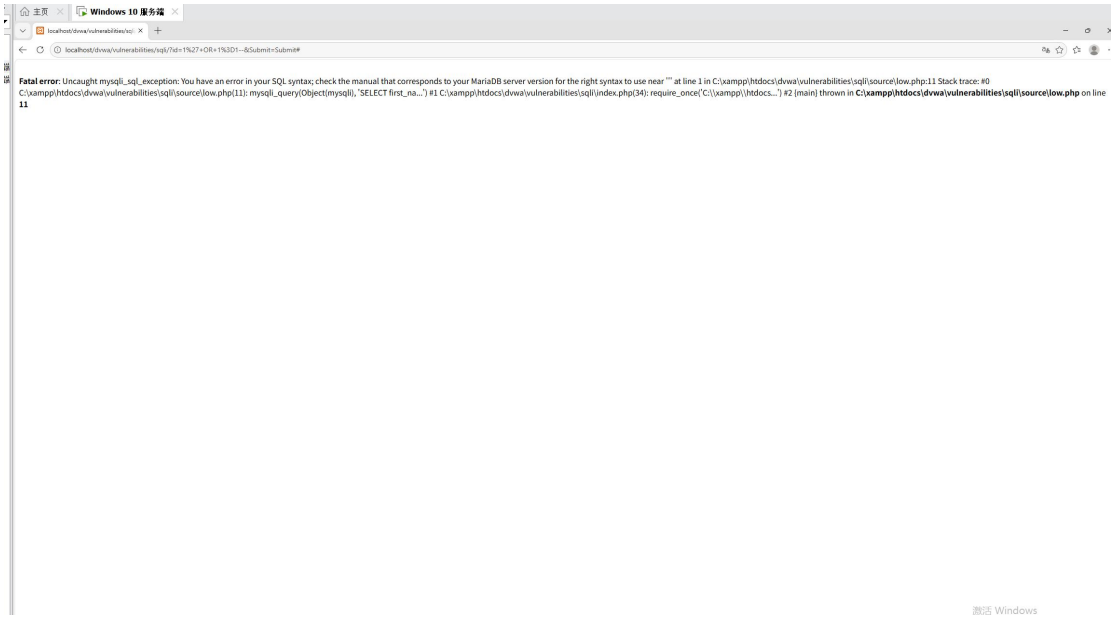

实验目标 / 页面：清晰展示了 DVWA 平台的 SQL Injection 模块，且左下角明确标注了 Security Level: low，完全符合 “DVWA SQL Injection Low” 的实验环境要求。

支撑的判断：这张截图可以直接支撑 “本次实验在无防护的 Low 级别 SQL 注入页面中进行” 的判断，是后续所有现象记录的基础。




输入内容：在 User ID 输入框中提交了合法参数 1。

返回结果：页面仅返回了与 ID=1 匹配的单条用户数据：
ID: 1
First name: admin
Surname: admin
支撑判断：这是正常查询的基准行为，输入合法参数时，SQL 语句只会返回匹配的单条数据，结果范围可控，没有超出业务逻辑。




输入内容：在 User ID 中输入了注入语句 1’ OR 1=1--
返回结果：页面返回了 Fatal error，提示 SQL 语法错误
支撑判断：该错误直接暴露了后台 SQL 语句的执行异常，说明输入被直接拼接到 SQL 语句中，查询结构被污染，存在 SQL 注入漏洞。

| 输入类型 | 输入内容 | 输入特征 |
|---|---|---|
| 普通输入 | 1 | 符合页面预期的合法数字 ID，无特殊字符，属于正常业务请求参数 |
| 异常输入 | 1’ OR 1=1-- | 包含 SQL 特殊字符（’、--）和逻辑语句，不符合正常输入格式，属于构造的恶意注入语句 |

3.返回结果的差异
普通输入（1）：页面仅返回与 ID=1 匹配的单条用户数据（ID: 1, First name: admin, Surname: admin），结果范围可控，符合正常业务查询逻辑。
异常输入（1’OR 1=1--）：页面返回 Fatal error 错误信息，提示 SQL 语法错误，暴露了后台数据库的执行细节，结果表现为非预期的系统报错。

4. 为什么这种差异不像 “正常查询变化”
正常业务查询中，结果只会随输入 ID 的不同返回对应单条数据，不会出现语法错误或超出业务逻辑的结果。本次实验中，异常输入导致的报错差异，是由于后台程序未对用户输入进行过滤或转义，恶意输入直接污染了 SQL 语句的语法结

构，改变了查询逻辑，属于 SQL 注入漏洞的典型表现，而非正常的业务查询变化。

# 任务 3：机制分析与差异比较

1. 输入如何进入查询结构（完整链路）
输入会遵循「输入 — 拼接 — 解析 — 执行 — 回显 / 行为差异」的链路进入查询结构：
输入：用户提交的内容被后台程序接收。
拼接：程序未对输入进行过滤，直接将其拼接到预设的 SQL 语句模板中，改变了原查询的结构。
解析：数据库将拼接后的字符串作为完整 SQL 语句进行语法解析。
执行：数据库执行被污染的 SQL 语句，返回对应数据或触发异常。
回显 / 行为差异：执行结果被页面回显，或表现为报错、返回数据量异常等行为差异。

2. 为什么根因是 “结构被污染” 而不是 “字符本身危险”
SQL 注入的核心问题并非输入中的特殊字符（如’、--）本身具有破坏性，而是这些字符被直接拼接进 SQL 语句后，改变了原查询的语法结构。
例如，输入’ OR 1=1--中的’并非 “危险字符”，但它会闭合原 SQL 语句中的字符串引号，OR 1=1 会改变查询条件的逻辑，--会注释掉后续语句，最终完全篡改了 SQL 的执行逻辑。如果程序对输入进行了转义或参数化查询，这些字符就无法污染查询结构，注入攻击也就无法实现。

3.Low 与 Medium 的差异在哪里

| 对比维度 | Low 级别 | Medium 级别 |
|---|---|---|
| 防护机制 | 无任何过滤，直接拼接用户输入 | 对输入进行了简单的转义处理（如转义单引号），并使用 GET 请求改为 POST 请求 |
| 注入难度 | 极低，直接构造恶意 SQL 即可 | 有所提升，需要绕过转义（如使用数字型注入、十六进制编码） |
| 核心差异 | 输入完全可控，直接污染查询结构 | 对特殊字符进行了基础过滤，但仍存在绕过可能，防护级别仅为基础 |

4. 普通 SQL 注入与 Blind SQL Injection（盲注）的差异在哪里

| 对比维度 | 普通 SQL 注入 | 盲注（Blind SQL Injection） |
|---|---|---|
| 信息回显方式 | 直接返回查询结果或报错信息，能直观看到数据 | 无直接数据回显，仅通过页面响应状态（如正常 / 异常、响应时间差异）判断 |

| 对比维度 | 普通 SQL 注入 | 盲注（Blind SQL Injection） |
| --- | --- | --- |
| 攻击方式 | 依赖错误回显或数据直接输出，可直接获取数据 | 依赖布尔判断（页面正常 / 异常）或时间延迟（如 sleep()函数）逐位猜解数据 |
| 攻击效率 | 较高，可直接获取数据 | 较低，需要多次请求进行猜解 |

5. 为什么盲注和普通注入根因相同，但证据形式不同
盲注和普通注入的根因完全相同：都是因为程序未对用户输入进行有效过滤，导致输入污染了 SQL 查询结构，执行了攻击者构造的恶意逻辑。两者的差异仅在于证据形式：
普通注入的“证据”是直接返回的数据或报错信息，直观可见。
盲注的“证据”是页面响应的状态差异（如布尔判断的页面变化、时间延迟），需要通过多次请求和逻辑判断来验证注入的存在，而非直接获取数据。

强制判断题解答
SQL 注入的根因更接近：B. 输入进入了不该进入的查询结构
理由：SQL 注入的本质是输入污染了 SQL 语句的语法结构，而非输入中的字符本身危险。危险字符只是实现结构污染的手段，核心问题是程序未将输入与查询结构分离。
Blind SQL Injection 和普通 SQL 注入的最大差异更接近：B. 证据形式不同
理由：两者的根因都是 SQL 查询结构被污染，只是普通注入能直接回显数据 / 错误作为证据，而盲注只能通过页面响应状态或时间差异间接验证，证据形式不同。

# 任务 4：SQL 注入防护清单与解释

1.参数化查询
解决什么问题：从根本上杜绝用户输入污染 SQL 查询结构的问题，将用户输入与 SQL 语句语法完全分离，输入始终作为参数值传递，无法改变查询逻辑。
控制哪一层：控制根因层，直接消除 SQL 注入产生的根本条件。
它不能替代什么：不能替代数据库的最小权限配置、错误处理机制，也不能防护业务逻辑漏洞或其他类型的攻击。
2. 输入校验
解决什么问题：对用户输入进行类型、格式、范围的校验与过滤，提前拦截恶意输入，减少注入攻击的成功概率。
控制哪一层：控制暴露面层，缩小可被利用的输入攻击面。
它不能替代什么：不能替代参数化查询，无法防御绕过校验的攻击（如编码绕过、白名单绕过），也不能解决已被允许的合法输入中隐藏的注入风险。
3. 错误处理
解决什么问题：避免向用户返回详细的数据库报错信息，防止攻击者通过错误回显获取数据库结构、字段名等敏感信息，降低盲注或错误注入的成功率。
控制哪一层：控制暴露面层，减少攻击者可利用的反馈信息。

它不能替代什么：不能阻止注入攻击本身，也不能防止攻击者通过布尔判断、时间延迟等盲注方式进行猜解。
4. 最小权限
解决什么问题：限制 Web 应用连接数据库时使用的账号权限，避免使用高权限账号（如 root），即使注入攻击成功，攻击者也无法执行超出权限范围的操作（如删除数据、修改表结构）。
控制哪一层：控制后果范围层，缩小攻击成功后的破坏范围。
它不能替代什么：不能阻止注入攻击的发生，也无法防止权限范围内的数据泄露或篡改。
5. 额外补充：WAF（Web 应用防火墙）
解决什么问题：通过规则匹配或机器学习识别 SQL 注入攻击特征，在请求到达应用前拦截恶意请求，提供额外的防护层。
控制哪一层：控制暴露面层，作为应用前的防护屏障。
它不能替代什么：不能替代应用层的参数化查询和输入校验，存在被绕过的风险（如变形注入、零日攻击）。

# 收口项 1：SQL 注入因果链/防护关系图

| 环节序号 | 环节名称 | 因果链描述 | 对应防护控制点 |
|---|---|---|---|
| 1 | 输入点 | 用户通过页面表单、URL 参数等入口提交数据，数据被后台接收。 | 输入校验、WAF 拦截 |
| 2 | 查询构造 | 后台程序未做过滤 / 转义，直接将用户输入拼接到 SQL 语句中，导致查询结构被污染。 | 参数化查询（核心防护） |
| 3 | 数据库解析 / 执行 | 数据库将拼接后的恶意 SQL 作为合法语句解析并执行，超出了正常业务的查询逻辑。 | 最小权限配置、数据库审计 |
| 4 | 结果差异或行为证据 | 执行结果异常，表现为报错回显、数据泄露、页面状态变化或响应时间差异，成为注入存在的证据。 | 错误处理、响应脱敏 |
| 5 | 根因判断 | 注入的根本原因是：用户输入污染了 SQL 查询结构，而非字符本身具有危险性。 | 无直接防护点，是对问题本质的总结 |

# 收口项 2：学生自评表

1. 我本章最清楚的一点是什么？
我最清楚的是 SQL 注入的根本成因：用户输入污染了 SQL 查询结构，而非特殊字符本身具有危险性。只要输入被直接拼接到 SQL 语句中，就存在被篡改查询

逻辑的风险，而参数化查询可以从根本上解决这个问题。

2. 我最容易混淆的一点是什么？
我最容易混淆的是普通 SQL 注入与盲注的差异。两者的根因完全相同，只是普通注入可以直接通过数据回显或报错获取信息，而盲注需要通过页面响应状态或时间延迟来间接判断，容易误以为它们是两种不同类型的漏洞。

3. 我本次作业中哪一段最需要教师复核？
本次作业中，我认为 “机制分析与差异比较” 部分最需要复核，尤其是对 Low 与 Medium 防护级别差异的理解，以及对 “结构被污染” 这一根因的阐述，不确定是否准确覆盖了课程的核心要求。

4. 如果重做一次，我会优先改哪一部分？
如果重做，我会优先完善 “防护清单” 部分，补充更多防护措施的实际应用场景，同时明确区分不同措施控制的是根因、暴露面还是后果范围，让逻辑更清晰、层次更分明。

# 收口项 3：AI 输出审核与人工复核记录

1. 本作业哪些内容使用过 AI 辅助
在本次 SQL 注入作业中，我使用 AI 辅助完成了以下内容：
各任务要点的结构化梳理与文字润色，包括 SQL 注入因果链、防护清单、机制分析等部分的框架搭建；
关键概念的通俗化解释，如 SQL 注入的根本成因、普通注入与盲注的差异等；
自评表与复核记录的参考内容撰写，提供了通用的表达框架。

2. 哪些判断是你自己复核后保留的
经过人工复核，以下核心判断我予以保留：
SQL 注入的根本原因是 “用户输入污染了 SQL 查询结构，而非特殊字符本身危险”；
参数化查询是控制注入根因的核心防护措施，输入校验、错误处理等措施控制的是暴露面；
普通 SQL 注入与盲注的根因相同，差异仅在于证据形式（直接回显 vs 间接判断）；
DVWA Low 级别无任何防护，Medium 级别仅对特殊字符做了基础转义处理。

3. 哪些内容你发现 AI 输出不准确并进行了修正
在复核过程中，我对 AI 输出的以下不准确内容进行了修正：
部分防护措施的层级分类错误，我重新明确了 “根因层、暴露面层、后果范围层” 的对应关系；
对盲注与普通注入的差异描述不够精准，补充了 “两者根因一致，仅证据形式不同” 的核心判断；
个别表述过于口语化，不符合课程作业的严谨性要求，我进行了专业术语规范和逻辑优化。

4. 你最终如何确认结论和证据是一一对应的

为确保结论与证据对应，我采用了以下方式进行确认：

对照 DVWA 实验现象，验证 “输入污染查询结构” 这一根因是否能解释普通输入与异常输入的结果差异；

对防护措施的描述，结合 “解决的问题 – 控制的层级 – 局限性” 的逻辑框架，检查是否与 SQL 注入的攻击链路匹配；

核对自评与复核内容，确保所有判断都有课程知识点或实验现象作为支撑，无无依据的主观臆断。